DEFENSIVE SECURITY · PROJECT 12

InternNova Cybersecurity Week 6

Mini Blue Team: Attack Simulation & Automated Detection Lab

Attack (8 simulated attacks) ▶ **Telemetry** (3 log sources, 394 events) ▶ **Detection** (7 rules, +1 after a miss) ▶ **Alert** (severity + technique ID) ▶ **Investigation** (timeline of the attack chain) ▶ **Response** (block, lock, clean, harden)

Why I chose this one

I wanted to answer the question every SOC analyst ends up asking: if someone really did attack this machine, would my detections notice? Purple teaming lets you test that instead of guessing. It also runs fully offline on a laptop, so it costs nothing.

Tools I used

Python 3 (re, json, datetime)
MITRE ATT&CK for naming techniques
My own log simulator + mini SIEM (normalize, rules, alerts)
Linux terminal, VS Code

How I built it

The lab has five parts: an attacker script, a Linux target (sshd, sudo, cron logs), a web target, a firewall, and a SIEM stand-in. Everything runs on one machine and only writes log files, so nothing real is attacked and nothing leaves the laptop. I used a script instead of full VMs to keep it light and finish in time. Wazuh is the next step, since that needs proper VMs.

The attacker script plays 8 attacks, mixed in with normal activity: priya mistyping her password three times, sanjay running *sudo bash*, random internet drops on the firewall. It also saves a ground-truth file of what I really "did". The detection script turns all three log formats into one event format, runs the rules, then checks every alert against the truth file.

Did my detections catch it?

first rule set: 7/8 | after tuning: 8/8 | false positives: 0

ID	ATT&CK	Simulated attack	Rule set v1	After tuning
A1	T1110.001	SSH brute force (fast)	CAUGHT	CAUGHT
A2	T1078	Valid account login after failures	CAUGHT	CAUGHT
A3	T1136.001	Backdoor user created	CAUGHT	CAUGHT
A4	T1548.003	Root shell via sudo	CAUGHT	CAUGHT
A5	T1053.003	Cron job persistence	CAUGHT	CAUGHT
A6	T1110.001	SSH brute force (low and slow)	MISSED	CAUGHT
A7	T1046	TCP port scan	CAUGHT	CAUGHT
A8	T1190	SQL injection	CAUGHT	CAUGHT

What I learned

- Reading ATT&CK technique IDs and matching real log lines to them
- Why you need harmless "normal" noise: it proves a rule doesn't over-fire
- Thresholds and time windows are exactly what attackers play with
- Lone alerts mean little, but sorted by time they tell a story
- Writing response steps straight from the evidence

Challenges I faced

My first rules missed an attack. The slow brute force tried once every ~5 minutes, so it never hit "10 in 5 minutes". I added a 1-hour window rule and reran everything.

Admins looked like attackers. sanjay really does use sudo bash and edit cron, so those rules needed an admin allowlist.

Three log formats. Each had its own time style, so I normalized to one event shape first.

Final outcome

The first rule set caught **7 of 8** attacks. After the fix it caught **8 of 8** with **0 false positives** on the normal activity I generated. The main intrusion shows up as one timeline from 10:30:27 to 10:41:08: brute force, login as admin, backdoor user, root shell, cron persistence. From it I wrote the response: block the IP, lock both accounts, remove the user and cron job, keys-only SSH. Honest limit: all data is simulated and it isn't Wazuh or Splunk yet, so porting these rules to one of them is what I'd do next.